



AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

Faculty of Science and Technology (FST)

Department of Computer Science (CS)

Undergraduate Program

Lab Manual

Course code and title: CSC 4162: Programming in Python

Credit: 3 credit hours (2 hours 20 minutes of Lab & 2 hours of theory per week)

Lab 01

Topics

- Installing Python.
- Start programming using the Python environment.

Background

There are different ways to write code in Python. In this course, we will see several options such as IDLE, Spyder, PyCharm, Jupyter Notebook, and Google Colab. However, we will mostly use IDLE and Google Colab to discuss our example codes.

Declaring Variables

Python has no command for declaring a variable. A variable is created the moment you first assign a value to it.

```
x = 5
y = "John"
print(x)
print(y)
```

Input and Output Values

Python uses the input() method to ask for a user input. The input() function returns the input value as a string.

```
username = input("Enter username:")
print("Username is:", username)
print("Username is: " + username)
print(type(username))
```

The Python print() function is often used to output variables.

List of problems

Problem 1. Demonstrate the installation steps of Python. The latest Python can be downloaded from this link - <https://www.python.org/downloads/>.

Problem 2. Introduction with IDLE and different coding environments such as PyCharm, Spyder, Jupyter Notebook, and Google Colab.

Problem 3. Explore IDLE and Google Colab using the previous basic programming knowledge.

- Perform mathematical operations.
- Take input values and print output values.

Lab 02

Topics

- Variables, Data types, and Operators
- Branching statements

Background

Variable Names

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)

```
myvar = "John"
my_var = "John"
_my_var = "John"
myVar = "John"
MYVAR = "John"
myvar2 = "John"
```

- A variable name cannot be any of the Python keywords.

Global Variables

Variables that are created outside of a function (as in all of the examples so far) are known as global variables. Global variables can be used by everyone, both inside of functions and outside.

```
x = "awesome"
def myfunc():
    print("Python is " + x)
myfunc()
```

Data Types

Python has the following data types built-in by default, in these categories:

Text Type:	str
Numeric Types:	int, float, complex
Sequence Types:	list, tuple, range
Mapping Type:	dict
Set Types:	set, frozenset
Boolean Type:	bool
Binary Types:	bytes, bytearray, memoryview
None Type:	NoneType

Example	Example (Specifying the data type)	Data Type
x = "Hello World"	x = str("Hello World")	str
x = 20	x = int(20)	int
x = 20.5	x = float(20.5)	float
x = 1j	x = complex(1j)	complex
x = ["apple", "banana", "cherry"]	x = list(("apple", "banana", "cherry"))	list
x = ("apple", "banana", "cherry")	x = tuple(("apple", "banana", "cherry"))	tuple
x = {"name" : "John", "age" : 36}	x = dict(name="John", age=36)	dict
x = {"apple", "banana", "cherry"}	x = set(("apple", "banana", "cherry"))	set
x = True	x = bool(5)	bool

Operators

Arithmetic Operators:

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y

*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

Comparison Operators:

Operator	Name	Example
==	Equal	x == y
!=	Not equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

Logical Operators:

Operator	Description	Example
and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 10)

Identity Operators:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

Membership Operators:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Operator Precedence:

Operator	Description
()	Parentheses
**	Exponentiation
+x -x ~x	Unary plus, unary minus, and bitwise NOT
* / // %	Multiplication, division, floor division, and modulus
+ -	Addition and subtraction
<< >>	Bitwise left and right shifts
&	Bitwise AND
^	Bitwise XOR
	Bitwise OR
== != > >= < <= is is not in not in	Comparisons, identity, and membership operators
not	Logical NOT
and	AND
or	OR

Branching statements

In python, if-else or if-elif-else statement is mostly used for branching.

```
a = 33
b = 200
if b > a:
    print("b is greater than a")
```

If-else (if only):

If-else:

If-elif-else (if-elif only):

If-elif-else:

```
a = 200
b = 33
if b > a:
    print("b is greater than a")
a = 33
b = 33
a = 200
b = 33
if b > a:
    print("b is greater than a")
elif a == b:
    print("a and b are equal")
else:
    print("a is greater than b")
```

Match Statement:

List of problems

Problem 1. Write a Python program to swap the values of two variables.

Problem 2. Write a program to compute the distance between two points, taking input from the

```
day = 4
match day:
    case 6:
        print("Today is Saturday")
    case 7:
        print("Today is Sunday")
    case _:
        print("This is Weekend")
```

user. [Distance formula = $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$]

Problem 3. Write a Program to check whether the given number is an even or odd number.

Problem 4. Take a character as input and determine whether it is a vowel. Therefore, print an appropriate message (vowel/not vowel).

Problem 5. Write a program to convert a student's marks of a course into a grade. Use the grading criteria of our university.

Lab 03

Topics

- Loops

Background

Python has two primitive loop commands:

- *while* loops
- *for* loops

```
fruits = ["apple", "banana", "cherry"]
for x in fruits:
    print(x)
```

```
for x in "banana":
    print(x)
```

```
fruits = ["apple", "banana", "cherry"]
for ind, val in enumerate(fruits):
    print(ind, val)
```

“for” loops

“for” loops (break & continue)

“for” loops (range() function)

“for” loops (else)

The else keyword in a for loop specifies a block of code to be executed when the loop is finished.

```
for x in range(6):
    print(x)
else:
    print("Finally finished!")
```

```
fruits = ["apple", "banana", "cherry"]
for x in range(2, 10, 3):
    print(x)
    if x == "banana":
        continue
    print(x)
```

The “else” block will NOT be executed if the loop is stopped by a “break” statement.

“for” loops (nested)

“while” loops

List of problems

```
adj = ["big", "tasty"]
fruits = ["apple", "banana", "cherry"]
for x in adj:
    i = 1
    while i < 6:
        print(i)
        i += 1
```

```
i = 1
while i < 6:
    print(i)
    if i == 3:
        break
    i += 1
```

```
i = 0
while i < 6:
    i += 1
    if i == 3:
        continue
    print(i)
```

```
i = 1
while i < 6:
    print(i)
    i += 1
else:
    print("i is no longer less than 6")
```

Problem 1. Write a program using a for loop that loops over a sequence.

Problem 2. Write a program using a while loop that asks the user for a number and prints a countdown from that number to zero.

Problem 3. Write a Python program to find the exponentiation of a number.

Problem 4. Find the sum of all the primes below 1000.

Problem 5. Print Fibonacci numbers below a given user input N.

Lab 05

Topics

- Dictionary
- Function

Background

Dictionaries are used to store data values in key:value pairs. A dictionary is a collection which is ordered*, changeable and does not allow duplicates.

```
thisdict = {  
    "brand": "Ford",  
    "electric": False,  
    "year": 1964,  
    "colors": ["red", "white", "blue"]  
}  
print(thisdict)
```

We can access the items of a dictionary by referring to its key name, inside square brackets. We can also use the get() method.

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
x = thisdict["model"]  
y = thisdict.get("model")  
print(x, y)
```

The update() method will update the dictionary with the items from the given argument.

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.update({"year": 2020})
```

You can loop through a dictionary by using a for loop.

```
for x in thisdict:
    print(x)
for x in thisdict:
    print(thisdict[x])
for x in thisdict.values():
    print(x)
for x in thisdict.keys():
    print(x)
for x, y in thisdict.items():
    print(x, y)
```

A dictionary can contain dictionaries, this is called nested dictionaries. You can loop through a dictionary by using the items() method like following.

```
myfamily = {
    "child1" : {
        "name" : "Emil",
        "year" : 2004
    },
    "child2" : {
        "name" : "Tobias",
        "year" : 2007
    },
    "child3" : {
        "name" : "Linus",
        "year" : 2011
    }
}
for x, obj in myfamily.items():
    print(x)

    for y in obj:
        print(y + ': ', obj[y])
```

In Python, a **function** is defined using the def keyword.

```
def my_function():
    print("Hello from a function")
my_function()
```

```
def my_function(fname, lname):  
    print(fname + " " + lname)  
my_function("Emil", "Refsnes")
```

Following is an example of using arguments.

Arbitrary Arguments

Keyword Arguments

Arbitrary Keyword Arguments

```
def my_function(*kids):  
    print("The last child is", kids[-1])  
def my_function(child3, child2, child1):  
    print("The last child is", child3)  
def my_function(**kid):  
    print("The last child is", kid["last child"])  
def my_function(country = "Norway"):  
    print("I am from " + country)  
my_function("Sweden")  
my_function("India")  
my_function()  
my_function("Brazil")
```

Default Parameter Value

Example of returning values

Function definitions cannot be empty, but if we for some reason have a function definition with no content, put in the pass statement to avoid getting an error.

```
def myfunction():  
    pass
```

```
def my_function(x):  
    return 5 * x  
print(my_function(3))  
print(my_function(5))  
print(my_function(9))
```

A **lambda** function is a small anonymous function. A lambda function can take any number of

```
x = lambda a, b : a * b
print(x(5, 6))
```

```
def myfunc(n):
    return lambda a : a * n
mydoubler = myfunc(2)
mytripler = myfunc(3)
print(mydoubler(11))
print(mytripler(11))
```

arguments, but can only have one expression.

List of problems

Problem 1. Write a Python program to combine two dictionaries by adding values for common keys.

```
d1 = {'a': 100, 'b': 200, 'c': 300}
d2 = {'a': 300, 'b': 200, 'd': 400}
Sample output: Counter({'a': 400, 'b': 400, 'd': 400, 'c': 300})
```

Problem 2. Write a Python program to count the frequency of values in a dictionary.

Original Dictionary: {'V': 10, 'VI': 10, 'VII': 40, 'VIII': 20, 'IX': 70, 'X': 80, 'XI': 40, 'XII': 20}
Count the frequency of the said dictionary: Counter({10: 2, 40: 2, 20: 2, 70: 1, 80: 1})

Problem 3. Write a Python function that checks whether a passed string is a palindrome.

Problem 4. Write a Python function that takes a list and returns a new list with distinct elements from the list.

```
Sample List : [1,2,3,3,3,3,4,5]
Sample output : [1, 2, 3, 4, 5]
```

Problem 5. Write a Python function that will take a list and print the counts of all distinct elements.

```
Sample List : [10,20,30,30,30,30,20,40]
Sample output : 10 => 1, 20 => 2, 30 => 4, 40 => 1
```

Problem 6. Write a Python lambda function to find if a given string starts with a given sub-string using Lambda.